



DESC



HISTORY



RULES



README



SETTINGS

① Codewriting

You're creating a new programming language with some exciting new features! Any programming language can check if two strings are matching, but you'd like yours to be able to check if they're *almost matching*. More specifically, we'll say two strings are *almost matching* if they're equal in length and all of their corresponding characters are the same except for one. For example, "cat" and "bat" are *almost matching*, but "cat" and "dog" are not.

For the sake of efficiency, you're planning on testing the feature by using a single string and comparing its substrings. Given a string s and an integer k , your task is to find the number of pairs of substrings of s that are *almost matching* but differ at their k^{th} character (0-based). It's necessary that the length of both substrings exceeds k (otherwise the strings wouldn't have a k^{th} character).

Also note that substrings are determined by their indices, so there could potentially be multiple instances of the same word. For example, in the word "ingratiating" the substring "ing"

main.cpp

```
1 int solution
2
3 }
4
```

TESTS

CUSTOM TESTS

| MORE



• ("ab" , "ac") - i = 0, j = 1, l = 2, m = 3 ;

• ("ab" , "ac") - i = 4, j = 5, l = 2, m = 3 .



main.cpp

```
1 int solution(s)
2
3 }
4
```

Input/Output

- [execution time limit] 0.5 seconds (cpp)

- [memory limit] 1 GB

- [input] string s

A string consisting only of lowercase English letters.

Guaranteed constraints:

$1 \leq s.length \leq 200$.

- [input] integer k

Guaranteed constraints:

$0 \leq k < s.length$.

- [output] integer

The amount of different pairs as described above.

TESTS

CUSTOM TESTS

MORE

 Codewriting

main.cpp

```
1  vector<int> s;  
2  
3  }  
4
```

Problem Description

Welcome to the magical world of Mario's adventure where Mario is standing on platform 0 and needs to reach platform $A+1$. But oh no, there's a wide river that's A units wide blocking his way. Mario can't walk on water, but he's got an amazing ability - he can jump up to B units at a time, either forward or backward.

On the river, there are various-sized wooden platforms floating on the river just like in Mario's games. In total there are M platforms and their sizes are stored in an integer array C . Mario can move these platforms around, but he can't change their order or let them overlap. Note that he should first move the platforms and then start his journey. Help Mario cross the river and reach platform $A+1$.

If it is impossible to reach $A+1$, your answer will be a vector containing -1 , acknowledging the challenge. Yet, if a solution exists, return a vector of size A representing the sequence of river cells. If cell i does

I

TESTS

CUSTOM TESTS

MORE

Example Output

Output1 :

[0, 1, 0, 2, 2, 0, 3]

Output2 :

[0, 0, 0, 0, 1, 1, 0, 0, 0, 0]

Example Explanation

For Input 1:

The lexicographically smallest possible answer is [0, 1, 0, 2, 2, 0, 3].

The sequence of jumps Mario performs is $0 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8$.

For Input2:

The lexicographically smallest possible answer is [0, 0, 0, 0, 1, 1, 0, 0, 0, 0].

The sequence of jumps you perform is $0 \rightarrow 5 \rightarrow 6 \rightarrow 11$

• [execution time limit] 0.5 seconds (cpp)



main.cpp

```
1  vector<
2
3  }
4
```

TESTS

CUSTOM TESTS

MORE

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.array.char field

A non-empty rectangular matrix of characters representing a field.

Guaranteed constraints:

$1 \leq \text{field.length} \leq 12$,

$1 \leq \text{field}[0].\text{length} \leq 12$.

- [output] array.integer

Array of two elements, where the first element is the minimum number of moves and the second one is the maximum number of moves required to fill the first column. It is guaranteed that the maximum number of moves doesn't exceed 12.

[C++] Syntax Tips

main.cpp

```
1 |vec  
2  
3 }  
4
```

TESTS

CUSTOM TESTS

Example

- For `crypt = ["SEND", "MORE", "MONEY"]`, the output should be

`solution(crypt) = 1`, because there is only one solution to this *cryptarithm*:

$0 = 0$, $M = 1$, $Y = 2$, $E = 5$, $N = 6$, $D = 7$, $R = 8$, and $S = 9$ ($9567 + 1085 = 10652$).

- For `crypt = ["GREEN", "BLUE", "BLACK"]`, the output should be

`solution(crypt) = 12`, because there are 12 possible valid solutions:

- $54889 + 6138 = 61027$
- $18559 + 2075 = 20634$
- $72449 + 8064 = 80513$
- $48229 + 5372 = 53601$
- $47119 + 5261 = 52380$
- $36887 + 4028 = 40915$
- $83447 + 9204 = 92651$
- $74665 + 8236 = 82901$
- $65884 + 7308 = 73192$
- $57883 + 6408 = 64291$
- $57881 + 6428 = 64309$

main.cpp

```
1 int solution(  
2  
3 }  
4
```

TESTS

CUSTOM TESTS

MORE

QUESTIONS

$$57883 + 6408 = 64291$$

$$57881 + 6428 = 64309$$

$$83441 + 9264 = 92705$$

- For `crypt = ["ONE", "TWO", "THREE"]`, the output should be `solution(crypt) = 0`, because there are no valid solutions.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.string crypt

Array of three non-empty strings containing only uppercase English letters.

Guaranteed constraints:

$$1 \leq \text{crypt}[i].\text{length} \leq 35$$

- [output] integer

The number of valid solutions.

[C++] Syntax Tips

**main.cpp**

```
1 int solution  
2  
3 }  
4
```

TESTS

CUSTOM TESTS

Codewriting

A *cryptarithm* is a mathematical puzzle where the goal is to find the correspondence between letters and digits such that the given arithmetic equation consisting of letters holds true.

Given a *cryptarithm* as an array of strings `crypt`, count the number of its valid solutions.

The solution is valid if each letter represents a different digit, and the leading digit of any multi-digit number is not zero.

`crypt` has the following structure: `[word1, word2, word3]`, which stands for the $\text{word1} + \text{word2} = \text{word3}$ *cryptarithm*.

Example

- For `crypt = ["SEND", "MORE", "MONEY"]`, the output should be

`solution(crypt) = 1`, because there is only one solution to this *cryptarithm*:

$0 = 0, M = 1, Y = 2, E = 5, N = 6, D = 7, R = 8$, and $S = 9$ ($9567 + 1085 = 10652$).



main.cpp

```
1 int solution
2
3 }
4
```

I

TESTS

CUSTOM TESTS

MORE

Example

- For

```
field = [['.', '.', '#', '#'],  
         ['#', '.', '.', '.'],  
         ['.', '.', '.', '.']]
```

the output should be

```
solution(field) = [4, 4]
```

We need to choose the first row once and the third row three times.



- For

```
field = [['.', '.', '#', '#'],  
         ['.', '.', '.', '#'],  
         ['.', '.', '.', '.']]
```

the output should be

```
solution(field) = [3, 6]
```

main.cpp

```
1 |vec  
2  
3 }  
4
```

TESTS

CUSTOM TEST

MORE



DESC

For example, if the board looks like this



```
...#..
...#..
.#....
```

blocks are denoted by '#' and empty cells are denoted by '.'

and you throw a block into the first or the second row from the top, it will end up like this:

```
...#..
...#..
.###..
```

Your task is to calculate the minimum and the maximum number of moves required to fill the first column of the board with blocks.

Example

- For

```
field = [['.', '#', '#'],
          ['#', '.', '.'],
          [., ., .]]
```

main.cpp

```
1 vector<int>
2
3 }
4
```



HISTORY



RULES



README



SETTINGS

TESTS

CUSTOM TESTS

MORE

Codewriting



main.cpp

```
1  |vector<i
2
3  }
4
```

You are given a rectangular board divided into a uniform grid (square cells). Some cells of the board are occupied with blocks, and others are empty. You are trying to add more and more blocks to the board, and your task is to fill the first column with them.

You can add a block to the field in the following way:

first, you choose the row index, then you throw the new block into the chosen row from the left. The block appears in the leftmost cell of the row and starts moving to the right, until it reaches another block or the end of the row. When that happens, the block starts falling down until it reaches another block or the last row.

For example, if the board looks like this

```
...#...
...#...
.#....
```

blocks are denoted by '#' and empty cells are denoted by '.'

and you throw a block into the first or the second row

TESTS

CUSTOM TESTS



Input Format

The first argument is an integer A

The second argument is an integer B

The third argument is an integer array C

Output Format

Return an integer array representing the answer.

Example Input

Input 1 :

A = 7

B = 2

C = [1, 2, 1]

Input 2 :

A = 10

B = 5

C = [2]

Example Output

Output1 :

main.cpp

```
1 vector<
2
3 }
4
```

TESTS

CUSTOM TESTS

- [input] string s

A string consisting only of lowercase English letters.

Guaranteed constraints:

$$1 \leq s.length \leq 200$$

- [input] integer k

Guaranteed constraints:

$$0 \leq k < s.length$$

- [output] integer

The amount of different pairs as described above.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloworld(string name) {
    cout << "This prints to the console who"
    return "Hello, " + name;
}
```

<<

main.cpp

```
1 int solution(string
2
3 }
4
```

TESTS

CUSTOM TESTS

MORE





BACK TO QUESTIONS

instances of the same word. For example, in the word "ingratiating" the substring "ing" beginning at index 0 is considered distinct from the one at index 9 (and there are also two distinct "ati" substrings).

Example

For $s = \text{"abacaba"}$ and $k = 1$, the output should be

$\text{solution}(s, k) = 8$.

Using i and j to represent the start and end indices of the first substrings, and l and m to represent the indices of the second substring, the 8 pairs are:

- ("aba" , "aca") - $i = 0, j = 2, l = 2, m = 4$;
- ("aba" , "aca") - $i = 4, j = 6, l = 2, m = 4$;
- ("aca" , "aba") - $i = 2, j = 4, l = 0, m = 2$;
- ("aca" , "aba") - $i = 2, j = 4, l = 4, m = 6$;
- ("ac" , "ab") - $i = 2, j = 3, l = 0, m$

<<

main.cpp

```
1 int solution(string s,  
2  
3 }  
4
```

RY

ES

i

ADME



TINGS

MORE

TESTS

CUSTOM TESTS



Question 2 of 4 Submitted

main.java

```
1  int[] solution(int a, int b, int[] c) {
2
3      int res[] = new int[a];
4      int m = c.length;
5      int total = 0;
6      for(int len : c) total += len;
7
8      int rem = a - total;
9      int curr = 0;
10     for(int i = 0; i < m; i++) {
11         int pos = curr + Math.min(rem, b - 1);
12         rem -= pos - curr;
13
14         for(int j = 0; j < c[i]; j++) {
15             res[pos + j] = i + 1;
16         }
17
18         curr = pos + c[i];
19     }
20     int lastpos = 0;
21     for(int i = 0; i < a; i++) {
22         if(res[i] != 0) {
23             if(i - lastpos > b) {
24                 return new int[]{-1};
25             }
26             lastpos = i + c[res[i] - 1] - 1;
27         }
28     }
29
30     if(a - lastpos > b) return new int[]{-1};
31     return res;
32 }
33
34 }
```

TESTS

CUSTOM TESTS

